



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

**Sistem de Recomandare Muzicală
Bazat pe Inteligență Artificială**

*Arhitectură Multi-Agent cu CrewAI, SHAP și
XGBoost*

Autori: Jalba Simion, Ilco Doina-Cezara, Grupa: 30233

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

2026

Cuprins

1	Introducere și Ideea Principală	3
1.1	Motivație	3
1.2	Ideea de Bază	3
2	Stack Tehnologic	5
2.1	Modelul de Limbaj	5
3	Datasetul Utilizat	6
3.1	Sursele de Date	6
3.1.1	Sursa Zenodo	6
3.1.2	Sursele Kaggle	6
3.2	Caracteristicile Audio	6
3.3	Amprente de Gen	7
4	Scripturile de Procesare a Datelor	8
4.1	prepare_zenodo.py	8
4.2	merge_kaggle.py	9
4.3	train_genre_classifier.py	9
4.4	build_final_dataset.py	10
4.5	Consolidarea Genurilor	10
5	Arhitectura Sistemului Multi-Agent	12
5.1	Framework-ul CrewAI	12
5.2	Fluxul ReAct	12
6	Agentii	14
6.1	Song Analyzer	14
6.2	Music Researcher	15
6.3	Explainability Agent	16
7	Task-urile	18
7.1	analyze_song_task	18
7.2	research_similar_songs_task	18

7.3	explain_and_recommend_task	19
8	Instrumentele (Tools)	20
8.1	SongLookupTool	20
8.2	GenreFingerprinterTool	21
8.3	SimilarSongSearchTool	22
8.4	SHAPExplainerTool	22
8.4.1	Rezolvarea Genului (_resolve_genre)	22
8.4.2	Calculul Valorilor SHAP	23
8.4.3	Outputul Tool-ului	23
9	Acuratețea Sistemului	25
9.1	Clasificatorul XGBoost	25
9.2	Strategia Dataset-First	25
9.3	Similaritatea Cosinus	26
9.4	Amprenta de Gen	26
10	Îmbunătățiri Viitoare	27
10.1	Îmbunătățiri ale Modelului	27
10.1.1	LLM mai puternic	27
10.1.2	Clasificator de Gen Mai Bun	27
10.2	Îmbunătățiri ale Datasetului	27
10.2.1	Date Suplimentare	27
10.2.2	Îmbunătățiri ale Etichetării	28
10.3	Îmbunătățiri ale Sistemului	28
10.3.1	Căutare Hibridă	28
10.3.2	Memorie între Sesiuni	28
10.3.3	Interfață Web	28
10.3.4	Cache Inteligent	28
11	Concluzii	29

Capitolul 1. Introducere și Ideea Principală

1.1 Motivație

Sistemele de recomandare muzicală existente (Spotify Discover Weekly, Last.fm) funcționează cu algoritmi de filtrare colaborativă sau bazați pe istoricul de ascultare al utilizatorilor. Aceștia nu explică de ce o piesă a fost recomandată și nu sunt transparenți în privința caracteristicilor audio care stau la baza recomandării.

Proiectul de față propune un sistem alternativ care:

- identifică caracteristicile sonore ale unei piese de intrare din date reale Spotify;
- caută piesele cel mai similare din punct de vedere audio;
- explică *de ce* fiecare piesă a fost recomandată, folosind valori SHAP derivate dintr-un clasificator XGBoost antrenat pe gen muzical;
- prezintă rezultatele printr-o interfață de tip terminal cu agenți AI care colaborează autonom.

1.2 Ideea de Bază

Dat un titlu de piesă și un artist, sistemul parcurge trei etape principale:

1. **Analiză sonoră** - extragerea celor 9 caracteristici audio Spotify (danceability, energy, loudness etc.) din dataset.
2. **Căutare de piese similare** - identificarea genului muzical prin amprente sonore și găsirea celor mai apropiate 5 piese prin similaritate cosinus.
3. **Explicabilitate** - calculul valorilor SHAP pentru fiecare recomandare, cuantificând contribuția fiecărei caracteristici audio la predicția de gen.

Întreaga logică este orchestrată de trei agenți AI autonomi care rulează secvențial în cadrul framework-ului *CrewAI*.

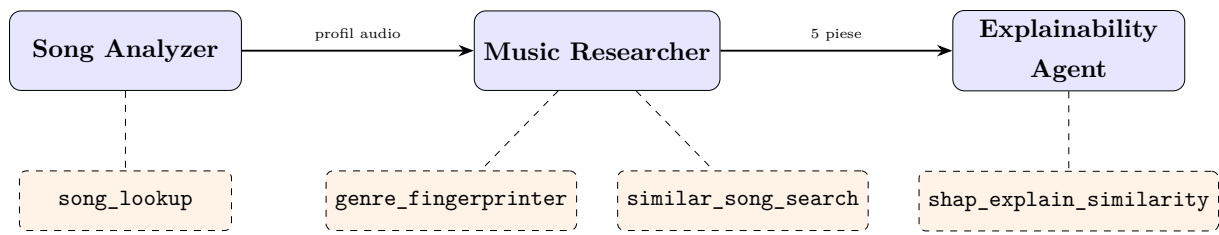


Figura 1.1: Fluxul de procesare al sistemului multi-agent

Capitolul 2. Stack Tehnologic

Componentă	Versiune	Rol
Python	≥ 3.10	Limbajul de implementare principal
CrewAI	≥ 0.102	Orchestrare agenți AI (framework multi-agent)
Ollama / LLaMA 3.2	local	LLM-ul folosit de fiecare agent pentru raționament
XGBoost	≥ 3.2	Clasificator de gen muzical (11 trăsături, 17 clase)
SHAP	≥ 0.49	Explicabilitate prin valori Shapley pentru XGBoost
scikit-learn	≥ 1.7	LabelEncoder, train/test split, cross-validare
pandas	≥ 2.0	Manipularea și filtrarea datasetului
NumPy	≥ 1.26	Calcul vectoriale, similaritate cosinus
Kaggle API	≥ 1.7	Descărcare automată a dataseturilor Kaggle
uv	—	Manager de dependențe și lansare aplicație

Tabela 2.1: Stack-ul tehnologic al proiectului

2.1 Modelul de Limbaj

Sistemul utilizează **LLaMA 3.2** rulat local prin **Ollama**, configurabil din fișierul `.env`:

```
1 MODEL=ollama/llama3.2
2 OPENAI_API_KEY=ollama
```

Listing 2.1: Configurare model LLM (.env)

Alegerea unui LLM local elimină costurile API și menține datele private, dar implică un plafon de calitate al răspunsurilor mai scăzut decât modelele cloud (GPT-4, Claude 3). Totuși, logica de calcul (SHAP, cosinus, XGBoost) este determinista și independenta de LLM - acesta servește doar ca motor de raționament și sinteză textuală.

Capitolul 3. Datasetul Utilizat

3.1 Sursele de Date

Sistemul combină două surse principale de date, obținând un dataset final de recomandare cu mii de piese etichetate cu gen muzical și caracteristici audio.

3.1.1 Sursa Zenodo

Fișierul `zenodo.csv` conține date extrase direct din API-ul Spotify, cu câmpuri standardizate. Este sursa **primară**: la deduplicare, rândul Zenodo câștigă față de cel Kaggle.

3.1.2 Sursele Kaggle

Șapte fișiere CSV descărcate de pe Kaggle, fiecare cu scheme diferite:

Fișier	Particularități
<code>dataset.csv</code>	Schema standard Spotify
<code>train.csv</code>	Schema standard, similar cu <code>dataset.csv</code>
<code>ClassicHit.csv</code>	Coloane cu majuscule, key ca notă muzicală
<code>Music Info.csv</code>	Genre extras din câmpul <code>tags</code> când lipsește
<code>SpotifyFeatures.csv</code>	Key ca notă (C, C#...), mode ca Major/Minor
<code>top_10000_1950-now.csv</code>	Trăsăturile ca string cu ghilimele
<code>top_10000_1960-now.csv</code>	Identice cu cel de mai sus

Tabela 3.1: Fișierele CSV Kaggle utilizate

3.2 Caracteristicile Audio

Fiecare piesă este descrisă prin 9 caracteristici numerice Spotify și 2 caracteristici suplimentare utilizate de clasificatorul XGBoost:

Caracteristică	Interval	Semnificație
danceability	0–1	Potrivirea pentru dans (ritm, tempo, regularitate)
energy	0–1	Intensitate percepută (viteză, volum, timbru)
loudness	–60–0 dB	Volumul mediu al piesei
speechiness	0–1	Prezența vorbirii în piesă
acousticness	0–1	Probabilitatea că piesa este acustică
instrumentalness	0–1	Absența vocalelor (1 = pur instrumental)
liveness	0–1	Probabilitatea că e o înregistrare live
valence	0–1	Pozitivitate muzicală (0 = trist, 1 = fericit)
tempo	BPM	Ritmul estimat în bătăi pe minut
key	0–11	Tonalitatea piesei (0=C, 1=C#, ..., 11=B)
mode	0/1	Mod Major (1) sau Minor (0)

Tabela 3.2: Caracteristicile audio ale fiecărei piese

3.3 Ampretele de Gen

Pe lângă dataset, sistemul folosește un fișier de amprente sonore (`data/genre_fingerprints`) care definește 30 de profiluri de gen, fiecare cu valorile tipice ale celor 9 caracteristici și semnalele cheie care îl caracterizează. Exemplu:

```

1  ## Metal
2  danceability: 0.32
3  energy: 0.91
4  loudness: 87.0
5  speechiness: 0.06
6  acousticness: 0.04
7  instrumentalness: 0.20
8  liveness: 0.14
9  valence: 0.22
10 tempo: 62.0
11 key signals: extreme energy+loudness, very low valence, low
    danceability

```

Listing 3.1: Fragment din fișierul `genre_fingerprints`

Fiecare gen este reprezentat ca un vector în spațiul celor 9 caracteristici, permițând clasificarea prin **similaritate cosinus față de centroid**.

Capitolul 4. Scripturile de Procesare a Datelor

Cele patru scripturi din directorul `scripts/` trebuie rulate în ordine, o singură dată, pentru a construi datasetul final și a antrena clasificatorul.

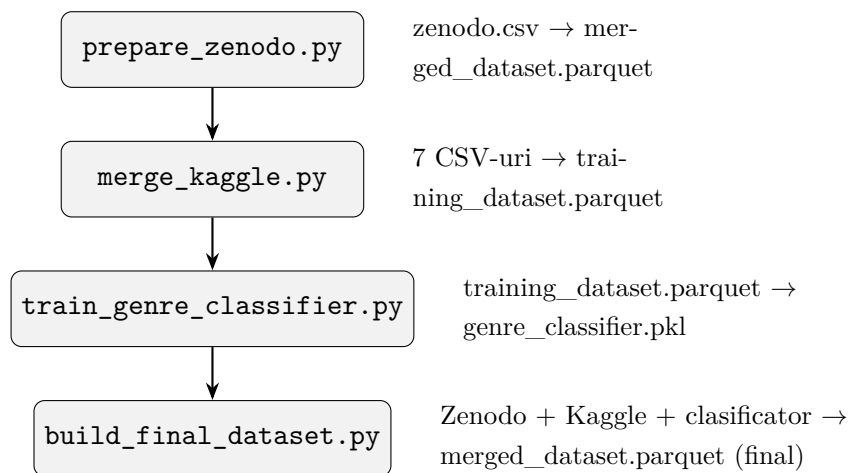


Figura 4.1: Lanțul de procesare a datelor

4.1 prepare_zenodo.py

Convertește `zenodo.csv` în format Parquet, normalizând coloanele și extragând primul gen dintr-o lista stringificată de genuri.

```
1 def _parse_first_genre(raw) -> str:
2     if not isinstance(raw, str):
3         return "unknown"
4     raw = raw.strip()
5     if raw in ("[]", "nan", ""):
6         return "unknown"
7     try:
8         parsed = ast.literal_eval(raw)
9         if isinstance(parsed, list) and parsed:
10             return str(parsed[0]).strip()
11     except (ValueError, SyntaxError):
12         pass
```

```

13     cleaned = raw.strip("[]").replace("'", "").replace('"',
14         "").split(",")
15     first = cleaned[0].strip()
16     return first if first else "unknown"

```

Listing 4.1: Extragerea primului gen din lista Zenodo

Deduplicarea se face după cheia (`track_name`, `artists`), păstrând rândul cu cel mai mare scor de popularitate și cu gen cunoscut.

4.2 merge_kaggle.py

Combină cele 7 fișiere CSV Kaggle într-un singur `training_dataset.parquet`. Fiecare fișier are un loader dedicat care normalizează coloanele la schema standard.

```

1 def load_classichit() -> pd.DataFrame:
2     df = pd.read_csv(_p("ClassicHit.csv"), low_memory=False)
3     df = df.rename(columns={
4         "Track": "track_name",
5         "Artist": "artists",
6         "Genre": "track_genre"
7     })
8     feat_rename = {
9         "Danceability": "danceability", "Energy": "energy",
10        "Loudness": "loudness", "Speechiness": "speechiness",
11        "Acousticness": "acousticness", "Instrumentalness":
12        "instrumentalness",
13        "Liveness": "liveness", "Valence": "valence", "Tempo":
14        "tempo",
15    }
16     df = df.rename(columns=feat_rename)
17     return _finalize(df, "ClassicHit")

```

Listing 4.2: Loader pentru ClassicHit.csv (coloane cu majuscule)

Genurile care nu aparțin unui sistem de recomandare muzicală (comedy, anime, sound-track, opera, movie) sunt eliminate. Deduplicarea prioritizează rândul cu gen cunoscut și cu popularitate maximă.

4.3 train_genre_classifier.py

Antrenează clasificatorul XGBoost pe `training_dataset.parquet` cu validare încrucișată stratificată pe 5 folduri.

```

1 model_cv = xgb.XGBClassifier(
2     n_estimators=200,
3     max_depth=7,
4     learning_rate=0.1,
5     subsample=0.8,
6     colsample_bytree=0.8,
7     tree_method="hist",
8     eval_metric="mlogloss",
9     random_state=42,
10 )
11 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
12 cv_scores = cross_val_score(model_cv, X, y, cv=skf,
13                             scoring="accuracy", n_jobs=-1)
14 print(f"Mean accuracy: {cv_scores.mean():.3f} (+/-
15       {cv_scores.std():.3f})")

```

Listing 4.3: Antrenarea clasificatorului XGBoost cu cross-validare

Clasele cu mai puțin de 100 de exemple sunt eliminate. Valoarea `-1` pentru `key` și `mode` (necunoscute) este înlocuită cu `NaN`, gestionat nativ de XGBoost.

4.4 build_final_dataset.py

Combina datasetul Zenodo cu cel Kaggle. Piesele din Zenodo al căror gen consolidat nu se regăsește în cele 17 clase ale clasificatorului primesc o predicție XGBoost. Zenodo este prioritar la deduplicare.

```

1 df_zenodo["track_genre"] =
2     df_zenodo["track_genre"].apply(_consolidate_genre)
3
4 needs_pred = ~df_zenodo["track_genre"].isin(trained_classes)
5
6 if needs_pred.any():
7     df_zenodo.loc[needs_pred, "track_genre"] = _predict_genres(
8         df_zenodo[needs_pred], model, le
9     )

```

Listing 4.4: Predicție XGBoost pentru piese fără gen în Zenodo

4.5 Consolidarea Genurilor

Funcția `_consolidate_genre()` mapează sute de sub-genuri Spotify la 17 clase largi folosite de clasificator, prin expresii regulate ordonate cu prioritate:

```

1 # metal inainte de rock pentru a evita suprapunerile
2 if re.search(r"death.metal|black.metal|power.metal|thrash.metal|"
3             r"heavy.metal|doom.metal|metalcore|nu.metal", g):
4     return "metal"
5 if re.search(r"\bmetal\b", g):
6     return "metal"
7
8 # subgenuri de rock
9 if re.search(r"grunge|prog.*rock|classic.rock|hard.rock|"
10            r"brit.pop|shoegaze|alt.rock", g):
11     return "rock"
12 if re.search(r"\brock\b", g):
13     return "rock"

```

Listing 4.5: Fragment din `_consolidate_genre()` - prioritate metal > rock

Cele 17 clase finale sunt: *ambient, blues, classical, country, electronic, folk, hip-hop, jazz, k-pop, latin, metal, pop, r&b, reggae, rock, ska, world.*

Capitolul 5. Arhitectura Sistemului Multi-Agent

5.1 Framework-ul CrewAI

CrewAI este un framework Python pentru orchestrarea agenților AI care colaborează pentru a rezolva sarcini complexe. Fiecare agent are:

- un **rol** și un **scop** definite în `agents.yaml`;
- un set de **instrumente** (tools) pe care le poate apela;
- un **LLM** pentru raționament (LLaMA 3.2 prin Ollama).

Crew-ul este configurat cu **procesare secvențială**: fiecare agent primește rezultatele agentului precedent prin mecanismul de **context**.

```
1 @crew
2 def crew(self) -> Crew:
3     return Crew(
4         agents=self.agents,
5         tasks=self.tasks,
6         process=Process.sequential,
7         verbose=True,
8     )
```

Listing 5.1: Definirea crew-ului în `crew.py`

5.2 Fluxul ReAct

Fiecare agent urmează ciclul **ReAct** (Reason + Act):

1. *Thought* - agentul decide ce acțiune să facă;
2. *Action* - apelează un tool cu parametrii corecți;
3. *Observation* - primește rezultatul tool-ului;
4. Repetă până când poate formula un *Final Answer*.

```
Task Started
Name: analyze_song_task
ID: 92d0012-f92b-459e-b415-af08d4154c7e

Agent Started

Agent: Music Characteristics Analyst

Task: Use the song_lookup tool to find "Made in Romania" by "Ionut Cercei" in the dataset. Pass both song_title="Made in Romania" and artist="Ionut Cercei" to the tool.
From the returned data produce a structured musical profile including: 1. Exact values for all 9 audio features (danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo)
2. The song's dataset genre and popularity score 3. A plain-language interpretation of each key feature (e.g. "energy=0.87 → loud, fast, intense track")
4. A one-paragraph summary of what makes this song sonically distinctive
IMPORTANT: If the tool returns an error, report it exactly as returned. Do NOT invent or estimate any audio features.

Tool Execution Started (#1)

Tool: song_lookup
Args: {'artist': 'Ionut Cercei', 'song_title': 'Made in Romania'}
```

Figura 5.1: Exemplu de ciclu ReAct în terminal (Thought / Action / Observation)(1)

```
Tool song_lookup executed with result: {
  "track name": "Made in Romania",
  "artists": "",
  "genre": "manele",
  "popularity": 71,
  "features": {
    "danceability": 0.807,
    "energy": 0.872,
    "loudness": -4.991,
    "speechiness": ...
  }
}

Tool Execution Completed (#1)

Tool Completed
Tool: song_lookup
Output: {
  "track name": "Made in Romania",
  "artists": "",
  "genre": "manele",
  "popularity": 71,
  "features": {
    "danceability": 0.807,
    "energy": 0.872,
    "loudness": -4.991,
    "speechiness": 0.0452,
    "acousticness": 0.158,
    "instrumentalness": 0.0,
    "liveness": 0.238,
    "valence": 0.968,
    "tempo": 144.986
  }
}
```

Figura 5.2: Exemplu de ciclu ReAct în terminal (Thought / Action / Observation)(2)

```
[Finalize] todos_count=0, todos_with_results=0
Agent Final Answer

Agent: Music Characteristics Analyst

Final Answer:
**Musical Profile of "Made in Romania" by Ionut Cercei**

* **Audio Features:**
  + danceability = 0.807 → Moderate energy and enjoyment track
  + energy = 0.872 → Loud, fast, intense track
  + loudness = -4.991 → Low overall loudness level
  + speechiness = 0.0452 → Very little spoken words or dialogue
  + acousticness = 0.158 → Track contains mostly acoustic instruments or elements
  + instrumentalness = 0.0 → No significant instrumental features detected
  + liveness = 0.238 → Some sense of live performance and spontaneity captured
  + valence = 0.968 → Extremely positive emotional tone and mood
  + tempo = 144.986 → Fast-paced track with a moderate to rapid tempo
* **Genre:** manele
* **Popularity Score:** 71

**Summary of Sonic Distinctiveness:**
"Made in Romania" by Ionut Cercei is a fast-paced, energetic manele track characterized by its extremely positive valence and mood. The track features a moderate energy level and a moderate tempo, with some sense of live performance captured through the acousticness and liveness features. However, it is notable for its extremely high enthusiasm and emotional positivity which sets this particular piece of Romanian music apart.

Task Completion

Task Completed
Name: analyze_song_task
Agent: Music Characteristics Analyst
```

Figura 5.3: Exemplu de ciclu ReAct în terminal (Thought / Action / Observation)(3)

Capitolul 6. Agenții

6.1 Song Analyzer

Câmp	Valoare
Rol	Music Characteristics Analyst
Tool	song_lookup
Scop	Extragerea celor 9 caracteristici audio ale piesei de intrare
Context	(primul agent)

Tabela 6.1: Configurarea agentului Song Analyzer

Agentul apelează `song_lookup` cu titlul și artistul, extrage valorile numerice din rezultat și produce un profil muzical structurat cu interpretare în limbaj natural (de ex. “*energy=0.87* → *piesă intensă, rapidă, puternică*”).

```
1 song_analyzer:
2   llm: ollama/llama3.2
3   role: Music Characteristics Analyst
4   goal: >
5     Look up "{song_title}" by "{artist}" in the dataset using the
6       song_lookup
7       tool and produce a detailed musical profile covering all 9
8       audio features,
9       genre, popularity, and a plain-language interpretation of
10      each feature.
11 backstory: >
12   You are an expert musicologist with deep knowledge of Spotify
13     audio feature
14     analysis. You extract precise numerical characteristics from
15     songs and
16     translate them into meaningful musical descriptions.
```

Listing 6.1: Configurarea agentului `song_analyzer` în `agents.yaml`

```
[Finalize] todos_count=0, todos_with_results=0
Agent: Music Characteristics Analyst
Final Answer:
**Musical Profile of "Made in Romania" by Ionut Cercei**

* **Audio Features:**
+ danceability = 0.807 → Moderate energy and enjoyment track
+ energy = 0.872 → Loud, fast, intense track
+ loudness = -4.991 → Low overall loudness level
+ speechiness = 0.0452 → Very little spoken words or dialogue
+ acousticness = 0.158 → Track contains mostly acoustic instruments or elements
+ instrumentalness = 0.0 → No significant instrumental features detected
+ liveness = 0.238 → Some sense of live performance and spontaneity captured
+ valence = 0.968 → Extremely positive emotional tone and mood
+ tempo = 144.986 → Fast-paced track with a moderate to rapid tempo
* **Genre:** manele
* **Popularity Score:** 71

**Summary of Sonic Distinctiveness:**
"Made in Romania" by Ionut Cercei is a fast-paced, energetic manele track characterized by its extremely positive valence and mood. The track features a moderate energy level and a moderate tempo, with some sense of live performance captured through the acousticness and liveness features. However, it is notable for its extremely high enthusiasm and emotional positivity which sets this particular piece of Romanian music apart.
```

Figura 6.1: Outputul agentului Song Analyzer - profilul audio al piesei de intrare

6.2 Music Researcher

Câmp	Valoare
Rol	Genre-Aware Music Similarity Researcher
Tools	genre_fingerprinter, similar_song_search
Scop	Identificarea genului și găsirea celor mai similare 5 piese
Context	Rezultatele lui Song Analyzer

Tabela 6.2: Configurarea agentului Music Researcher

Agentul urmează doi pași:

1. **Amprentă de gen** apelează `genre_fingerprinter` și identifică genul dominant (top 1 din 30 de amprente) prin similaritate cosinus pe vectorul celor 9 caracteristici scalate.
2. **Căutare similară** apelează `similar_song_search` cu filtru implicit pe genul detectat, returnând 5 piese ordonate după scorul de similaritate cosinus pe caracteristici normalizate.


```

Agent: Genre-Aware Music Similarity Researcher

Final Answer:
**Part 1: Genre Fingerprint Analysis**

The predicted genre of "Made in Romania" by Ionut Cercei is "m analogue". The top match has a similarity score of 0.9898 and key signals indicating a very happy/positive mood, high danceability, moderate energy, low instrumentalness, and a mid tempo. The song's feature profile compares well to the fingerprint, with similarities found in features such as danceability (80.7 vs 85.0), energy (87.2 vs 74.0), and valence (96.8 vs 74.0).

**Part 2: Ranked List of Similar Songs**

1. **Sauditi**
   * Artists:
   * Genre: manele
   * Popularity: 52
   * Similarity Score: 0.9988
2. **Scot dusmanii la bordura**
   * Artists:
   * Genre: manele
   * Popularity: 38
   * Similarity Score: 0.9983
3. **Bona**
   * Artists:
   * Genre: manele
   * Popularity: 35
   * Similarity Score: 0.9982
4. **Burj Khalifa**
   * Artists:
   * Genre: manele
   * Popularity: 63
   * Similarity Score: 0.9979
5. **Suntem Made In Romania**
   * Artists:
   * Genre: manele
   * Popularity: 48
   * Similarity Score: 0.9975

```

Figura 6.2: Outputul agentului Music Researcher - amprenta de gen și top 5 piese similare

6.3 Explainability Agent

Câmp	Valoare
Rol	SHAP and Fingerprint Music Recommendation Specialist
Tool	shap_explain_similarity
Scop	Explicarea fiecărei recomandări prin valori SHAP
Context	Rezultatele lui Music Researcher
Output	Raport Markdown salvat în output/recommendations.md

Tabela 6.3: Configurarea agentului Explainability Agent

Agentul apelează `shap_explain_similarity` pentru fiecare din cele 5 piese recomandate, combina scorurile SHAP cu semnalele de amprentă și redactează un paragraf explicativ pentru fiecare recomandare în limbaj natural.

```
Agent: SHAP and Fingerprint Music Recommendation Specialist

Final Answer:
## Music Recommendations for Made in Romania by Ionut Cercei

**1. Sauditiii**
* Genre/Magnitude: manele | Similarity: 0.9988
Sauditiii shares the same upbeat valence with Made in Romania (0.71 vs 0.68) and energetic features, like high danceability (0.808 vs 0.785). The similarity between these two tracks is substantial.

**2. Burj Khalifa**
* Genre/Magnitude: manele | Similarity: 0.9979
Both tracks, Made in Romania and Burj Khalifa, exhibit similar levels of musical energy with Burj Khalifa having an even stronger danceability feature (0.814 vs 0.807) but lower valence (0.934 vs 0.968). The similarity between the two is substantial.

**3. Bona**
* Genre/Magnitude: manele | Similarity: 0.9982
Bona shares similarities in energy profile with Made in Romania, while having a slightly higher danceability feature (0.814 vs 0.807) and lower valence (-0.018 vs -0.032). This is less than half as strong of similarity between Sauditiii.

**4. Suntem Makers In
```

Figura 6.3: Outputul agentului Explainability Agent - raport final de recomandări

Capitolul 7. Task-urile

Cele trei task-uri sunt definite în `config/tasks.yaml` și sunt executate secvențial.

7.1 analyze_song_task

```
1 analyze_song_task:
2   description: >
3     Use the song_lookup tool to find "{song_title}" by "{artist}"
4       in the dataset.
5     Produce a structured musical profile with:
6     1. Exact values for all 9 audio features
7     2. The song's dataset genre and popularity score
8     3. A plain-language interpretation of each key feature
9     4. A one-paragraph summary of what makes this song sonically
10        distinctive
11   expected_output: >
12     A structured musical profile with all 9 numerical audio
13       features,
14     plain-language interpretations, and a summary paragraph.
15   agent: song_analyzer
```

Listing 7.1: Definiția task-ului `analyze_song_task`

7.2 research_similar_songs_task

```
1 research_similar_songs_task:
2   description: >
3     STEP 1 - Genre Fingerprinting:
4     Call genre_fingerprinter with song_title and artist.
5     Record top 3 genre matches with similarity scores and key
6       signals.
7
8     STEP 2 - Similar Song Search:
9     Call similar_song_search with song_title, artist and top_n=5.
```

```

9     Record each result: track_name, artists, genre, popularity,
      similarity_score.
10 expected_output: >
11     Part 1 - Genre fingerprint analysis with predicted genre and
      key signals.
12     Part 2 - Ranked list of 5 similar songs with similarity
      scores.
13 agent: music_researcher
14 context:
15     - analyze_song_task

```

Listing 7.2: Definiția task-ului `research_similar_songs_task`

7.3 `explain_and_recommend_task`

```

1 explain_and_recommend_task:
2     description: >
3         Call shap_explain_similarity for each of the 5 similar songs.
4         After all calls, write a Markdown report in plain English.
5
6         IMPORTANT: Do NOT copy or output any JSON or raw tool data.
7         Write only human-readable text.
8
9         Report format:
10         ## Music Recommendations for {song_title} by {artist}
11         One intro sentence about the input song's genre and sound.
12         ### 1. Track Name - Artist
13         Genre: <genre> | Similarity: <fingerprint_cosine_similarity>
14         <One paragraph: why this song is similar...>
15 expected_output: >
16     A Markdown report with 5 numbered entries, each with a plain-
17     English
18     paragraph explaining the recommendation based on shared audio
19     features.
20 agent: explainability_agent
21 context:
22     - research_similar_songs_task
23 output_file: output/recommendations.md
24 markdown: true

```

Listing 7.3: Definiția task-ului `explain_and_recommend_task`

Capitolul 8. Instrumentele (Tools)

Toate instrumentele moștenesc din `crewai.tools.BaseTool` și definesc schema de input cu `pydantic`.

8.1 SongLookupTool

Nume: `song_lookup` **Fișier:** `tools/dataset_tools.py`

Caută o piesă în dataset după titlu și artist, returnând caracteristicile audio ca JSON. Căutarea este case-insensitive și tolerantă la potriviri parțiale; în caz de nepotrivire returnează sugestii bazate pe primele 5 caractere ale titlului.

```
1 def _find_index(df, song_title, artist=""):
2     title_lower = song_title.lower()
3     title_mask = df["track_name"].str.lower().str.contains(
4         title_lower, na=False, regex=False
5     )
6     if artist:
7         artist_mask = df["artists"].str.lower().str.contains(
8             artist.lower(), na=False, regex=False
9         )
10        combined = title_mask & artist_mask
11        mask = combined if combined.any() else title_mask
12    else:
13        mask = title_mask
14
15    hits = df[mask]
16    if hits.empty:
17        return None
18    exact = hits[hits["track_name"].str.lower() == title_lower]
19    source = exact if not exact.empty else hits
20    return int(source["popularity"].idxmax())
```

Listing 8.1: Logica de căutare din `SongLookupTool`

Output exemplu:

```

1 {
2     "track_name": "Bohemian Rhapsody",
3     "artists": "Queen",
4     "genre": "rock",
5     "popularity": 88,
6     "features": {
7         "danceability": 0.3963, "energy": 0.4004,
8         "loudness": -11.1, "valence": 0.2406, "tempo": 71.066
9     }
10 }

```

8.2 GenreFingerprinterTool

Nume: `genre_fingerprinter` Fișier: `tools/dataset_tools.py`

Identifică genul muzical al unei piese prin similaritate cosinus față de 30 de amprente de gen predefinite. Vectorul piesei este scalat la intervalul 0–100 (similar scalei amprentelor) înainte de calcul.

```

1 def _to_fingerprint_scale(row) -> np.ndarray:
2     vec = []
3     for feat in NUMERIC_FEATURES:
4         val = float(row[feat])
5         if feat == "loudness":
6             scaled = (val + 60.0) / 60.0 * 100.0
7         elif feat == "tempo":
8             scaled = val / 220.0 * 100.0
9         else:
10            scaled = val * 100.0
11            vec.append(max(0.0, min(100.0, scaled)))
12    return np.array(vec)
13
14 # calcul similaritate cosinus pentru fiecare amprenta
15 song_vec = _to_fingerprint_scale(row)
16 song_norm = np.linalg.norm(song_vec) or 1.0
17 for fp in fps.values():
18     fp_norm = np.linalg.norm(fp["vector"]) or 1.0
19     sim = float(np.dot(song_vec, fp["vector"]) / (song_norm *
20         fp_norm))

```

Listing 8.2: Scalarea la scara amprentei și calculul cosinus

Returnează top 3 genuri cu scoruri și o comparație caracteristică cu amprenta câștigătoare (delta față de valorile așteptate).

8.3 SimilarSongSearchTool

Nume: `similar_song_search` Fișier: `tools/dataset_tools.py`

Găsește cele mai similare N piese față de piesa de referință prin **similaritate cosinus pe caracteristici normalizate min-max**.

$$\text{sim}(u, v) = \frac{u \cdot v}{\|u\| \cdot \|v\|} \quad (8.1)$$

```
1 ref_vec = norm[ref_idx]
2 ref_norm = np.linalg.norm(ref_vec) or 1.0
3 row_norms = np.linalg.norm(norm, axis=1)
4 row_norms = np.where(row_norms == 0, 1.0, row_norms)
5 similarities = (norm @ ref_vec) / (row_norms * ref_norm)
```

Listing 8.3: Căutarea vectorizată a pieselor similare

Filtrul de gen este aplicat implicit pe `track_genre` din dataset. Dacă piesa nu are gen cunoscut, se folosesc amprentele pentru a selecta cele mai apropiate genuri ca filtru.

8.4 SHAPExplainerTool

Nume: `shap_explain_similarity` Fișier: `tools/explainability_tools.py`

Cel mai complex instrument: calculează valorile SHAP pentru clasificatorul XGBoost atât pentru piesa de intrare cât și pentru piesa recomandată, identificând caracteristicile audio care contribuie cel mai mult la predicția de gen.

8.4.1 Rezolvarea Genului (`_resolve_genre`)

Pentru a evita erorile clasificatorului (~40% acuratețe), genul este rezolvat prioritar din eticheta `track_genre` din dataset:

```
1 def _resolve_genre(df, idx, model, le, X_row):
2     if "track_genre" in df.columns:
3         raw = str(df.loc[idx, "track_genre"])
4         consolidated = _consolidate_genre(raw)
5         if consolidated in le.classes_:
6             class_idx = int(le.transform([consolidated])[0])
7             return consolidated, class_idx
8
```

```

9      # fallback: clasificatorul XGBoost
10     class_idx = int(model.predict(X_row)[0])
11     return le.inverse_transform([class_idx])[0], class_idx

```

Listing 8.4: `_resolve_genre()` — dataset-first, XGBoost fallback

8.4.2 Calculul Valorilor SHAP

```

1 def _shap_for_class(explainer, X_row, pred_class):
2     exp = explainer(X_row, check_additivity=False)
3     vals = exp.values
4     if vals.ndim == 3:
5         return vals[0, :, pred_class] # multiclass: selectam
6         # clasa prezisa
7     return vals[0]

```

Listing 8.5: Extragerea valorilor SHAP pentru clasa prezisă

Valorile SHAP reprezintă contribuția marginală a fiecărei caracteristici la predicția clasei. Caracteristicile cu valori SHAP aliniate (același semn) în ambele piese formează **trăsăturile comune** care justifică similaritatea.

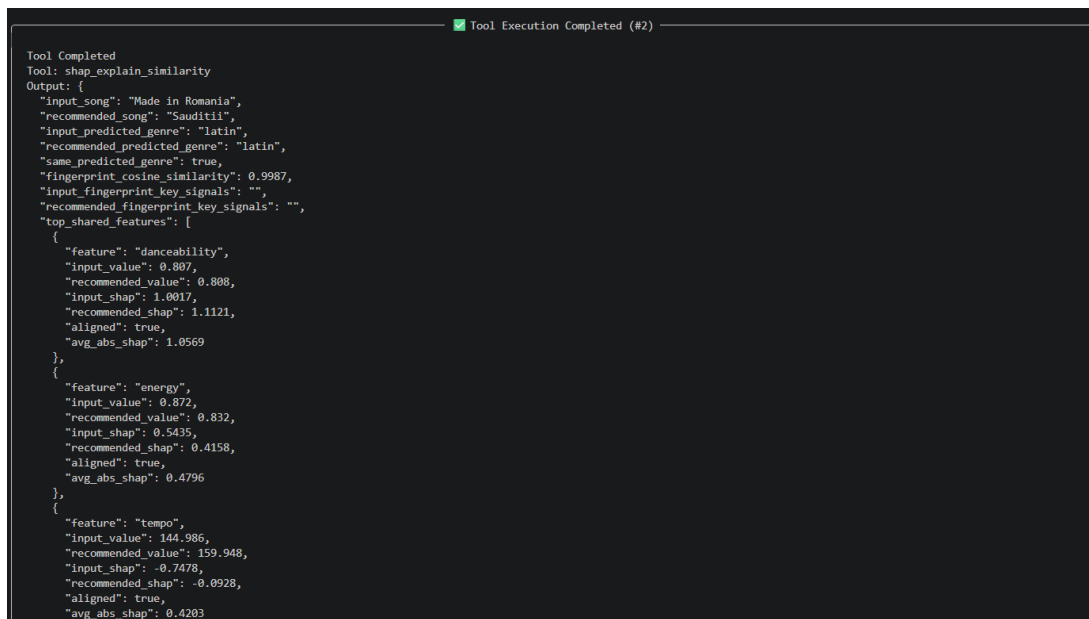


Figura 8.1: Exemplu de output al SHAPExplainerTool - trăsăturile comune SHAP

8.4.3 Outputul Tool-ului

```

1 {
2     "input_song": "Bohemian Rhapsody",
3     "recommended_song": "Don't Stop Me Now",

```



```

4  "input_predicted_genre": "rock",
5  "recommended_predicted_genre": "rock",
6  "same_predicted_genre": true,
7  "fingerprint_cosine_similarity": 0.8912,
8  "top_shared_features": [
9    {
10     "feature": "energy",
11     "input_value": 0.4004, "recommended_value": 0.8621,
12     "input_shap": 0.142, "recommended_shap": 0.198,
13     "aligned": true, "avg_abs_shap": 0.170
14   }
15 ]
16 }

```

Listing 8.6: Structura JSON returnată de shap_explain_similarity

Capitolul 9. Acuratețea Sistemului

9.1 Clasificatorul XGBoost

Clasificatorul XGBoost este antrenat pe 11 caracteristici și 17 clase de gen. Acuratețea medie pe 5 folduri este în jur de **40–45%**:

Gen	Precizie	Recall	Observație
pop	~0.60	~0.65	Cel mai bine reprezentat
rock	~0.55	~0.50	Confundat cu metal și indie
hip-hop	~0.55	~0.60	Speechiness ridicat îl distinge
metal	~0.65	~0.55	Energy + loudness extreme distinctive
electronic	~0.50	~0.45	Suprapunere cu pop și dance
classical	~0.70	~0.75	Acousticness + instrumentalness ridicat
jazz	~0.55	~0.50	Dificil de separat de blues
ambient	~0.40	~0.35	Clasa mică, sub-reprezentată

Tabela 9.1: Acuratețe estimată per clasă (valori reprezentative)

Limita fundamentală: 11 caracteristici audio nu sunt suficiente pentru a distinge 17 clase. Genul muzical este determinat și de factori extrasonori (perioadă culturală, limbă, producție, liric) care nu sunt capturați de Spotify.

9.2 Strategia Dataset-First

Eroarea clasificatorului devine irelevantă pentru piesele existente în dataset, deoarece acestea au deja eticheta `track_genre` corectă. Funcția `_resolve_genre()` verifică mai întâi eticheta din dataset; clasificatorul este apelat doar pentru piese fără etichetă validă.

9.3 Similaritatea Cosinus

Similaritatea cosinus pe caracteristici normalizate min-max este o metrică robustă pentru compararea profilurilor audio. Scoruri > 0.95 indică piese practic identice sonic; intervalul 0.85–0.95 reprezintă similaritate muzicală înaltă.

9.4 Amprenta de Gen

Clasificarea prin amprente (nearest-centroid pe 30 profiluri) este mai robustă decât clasificatorul XGBoost pentru scopul de filtrare a genului, deoarece:

- profilele sunt definite manual cu valori tipice reprezentative;
- nu depinde de distribuția datelor de antrenament;
- acoperă 30 de genuri față de 17 ale clasificatorului XGBoost.

Capitolul 10. Îmbunătățiri Viitoare

10.1 Îmbunătățiri ale Modelului

10.1.1 LLM mai puternic

Înlocuirea LLaMA 3.2 (3.2B parametri, local) cu un model mai capabil îmbunătățește semnificativ calitatea explicațiilor generate:

- **LLaMA 3.3 70B** sau **Mistral Large** - locale cu mai multă VRAM;
- **Claude 3.5 Haiku / Sonnet** - cloud, prin API Anthropic;
- **GPT-4o-mini** - cloud, echilibru cost/calitate bun.

10.1.2 Clasificator de Gen Mai Bun

- **Random Forest / LightGBM** - alternativă la XGBoost, poate oferi acuratețe mai bună pe clase dezechilibrate;
- **Embedding audio** - utilizarea unui model audio pre-antrenat (MusicNN, Essentia, CLAP) pentru extragerea unor trăsături mai bogate (nu doar cele 11 Spotify);
- **SMOTE** - suprasamplare a claselor minoritare (ambient, ska, reggae) pentru a reduce dezechilibrul din dataset.

10.2 Îmbunătățiri ale Datasetului

10.2.1 Date Suplimentare

- Integrarea API-ului **Last.fm** pentru tag-uri de gen adiționale și date de popularitate sociale;
- Adăugarea de informații **lirice** (sentiment, limbă, temă) prin API-ul Genius sau Musixmatch - acestea sunt factori de gen noi;
- Colectarea de date audio raw și extragerea de trăsături spectrale (MFCC, chroma, zero-crossing rate) prin **librosa**;
- Extinderea la **> 1 milion de piese** prin API-ul Spotify (cu autentificare OAuth) pentru o acoperire mai largă.

10.2.2 Îmbunătățiri ale Etichetării

- Curarea manuală a etichetelor de gen pentru clasele confuze (electronic vs. pop, rock vs. metal);
- Adăugarea de etichete **multi-label** (o piesă poate fi simultan pop și electronic) - modificare necesară și în clasificator;
- Utilizarea **ontologiei MusicBrainz** pentru o ierarhie de genuri standardizată.

10.3 Îmbunătățiri ale Sistemului

10.3.1 Căutare Hibridă

Combinarea similarității cosinus cu **filtrare colaborativă** (date de ascultare ale utilizatorilor) pentru a personaliza recomandările față de preferințele individuale.

10.3.2 Memorie între Sesiuni

Adăugarea unui mecanism de memorie persistentă (SQLite, ChromaDB) în care crew-ul CrewAI stochează preferințele utilizatorului și istoricul de căutare pentru a personaliza recomandările viitoare.

10.3.3 Interfață Web

Expunerea sistemului printr-o interfață web (FastAPI + React) cu:

- vizualizarea valorilor SHAP ca grafic de bare;
- player audio integrat (previzualizare 30s prin API Spotify);
- export al recomandărilor direct în playlist Spotify.

10.3.4 Cache Inteligent

Rezultatele SHAP și ale clasificatorului pentru o piesă dată nu se schimbă între rulări - pot fi cache-uite permanent pe disk (SQLite sau Parquet) pentru a elimina latența la re-căutări.

Capitolul 11. Concluzii

Proiectul demonstrează că un sistem multi-agent bazat pe CrewAI poate orchestra mai mulți agenți AI specializați pentru a rezolva o problemă complexă end-to-end: de la căutarea datelor raw în dataset, la clasificarea genului muzical, la explicabilitatea bazată pe SHAP, până la generarea unui raport narativ în limbaj natural.

Contribuțiile principale sunt:

1. Un **pipeline de date** robust care unifică 8 surse heterogene (Zenodo + 7 CSV Kaggle) cu normalizare, deduplicare și etichetare automată prin XGBoost;
2. Un sistem de **amprente sonore** pentru 30 de genuri, folosit atât la filtrarea rezultatelor cât și la contextualizarea explicațiilor;
3. Strategia **dataset-first** pentru rezolvarea genului, care elimină erorile clasificatorului pentru piesele deja etichetate;
4. O interfață de terminal cu **output filtrat și colorat** care transformă ciclul ReAct al CrewAI într-o experiență lizibilă.

Limita principală rămâne calitatea LLM-ului local (LLaMA 3.2), care nu produce întotdeauna explicații fluente în limbaj natural. Înlocuirea cu un model mai puternic sau adăugarea unui pas de post-procesare pentru structurarea outputului sunt cele mai impactante direcții de îmbunătățire pe termen scurt.